

Chapter 11 — From fundamentals to advanced annotation

Chapter 4 covered annotation modalities, workflows, tools, and quality control

Learning objectives

- By the end of this part

Shared labeling bottleneck

- Manual labels are slow, expensive
- Chapter 4 already identified that bottleneck
- Advanced strategies attack it from different angles

Four strategies at a glance

- Active learning chooses which unlabeled items humans should label next
- Weak supervision proposes training labels from heuristics and distant sources
- Self-supervised learning postpones task labels by pretraining on unlabeled structure
- Hybrid crowd-expert pipelines assign routine items to crowds and reserve experts for gold

Table 11.1 — What each strategy optimizes

- The chapter comparison table contrasts who provides labels, when each strategy fits
- Active learning optimizes which items humans label when a seed model can rank uncertainty
- Weak supervision optimizes scale from cheap sources when domain rules exist

Table 11.1 — Self-supervision and hybrid pipelines

- But still needs labeled data for the final task
- Hybrid crowd-expert staffing optimizes the cost-quality mix when the schema is partly

Complementary, not exclusive

- Teams often combine strategies
- Synthetic companions from Chapter 10 can still fill rare classes

Takeaways

- Four strategies share one goal: reduce costly expert labeling without abandoning quality
- Active learning queries humans selectively
- The remainder of the chapter develops each strategy in turn

Next

- Complete the quiz for this part
- Entropy sampling, with examples from vision and medical imaging

Learning objectives

- By the end of this part

The iterative feedback loop

- Active learning starts from a small labeled seed set
- The cycle repeats until performance or budget stops the loop
- The goal is better generalization with fewer total labels than random annotation

Query strategies overview

- A query strategy defines how the model ranks unlabeled items for human labeling
- Uncertainty sampling targets low-confidence predictions near decision boundaries
- Margin sampling ranks by the gap between the top two class probabilities
- Entropy sampling ranks by predictive entropy over the full class distribution
- Strategies can be combined or adapted to the task

Example 11.1 — Uncertainty Sampling for Cat-Dog Images

- Example 11.1 — hands-on module
- Example 11.1 shows uncertainty sampling on a binary vision task
- An image with predicted probabilities near 0.51 and 0.49 for cat versus dog is queued for
- Explore the chapter example module
- View files: ``modules/chapter11/example1/``

Example 11.1 — listing

```
"""Example 11.1 – uncertainty sampling queues near-tie binary
predictions."""

from __future__ import annotations

def uncertainty_score(p_positive: float) -> float:
    """Return distance from a 0.5 decision boundary (lower => more
    uncertain)."""
    return abs(p_positive - 0.5)

def main() -> None:
    """Rank unlabeled images by binary prediction uncertainty."""
    pool: list[tuple[str, float]] = [
        ("img_clear_dog.jpg", 0.92),
        ("img_clear_cat.jpg", 0.11),
        ("img_clear_person.jpg", 0.51)
```

Example 11.2 — Margin Sampling in Multi-Class Tasks

- Example 11.2 — hands-on module
- Example 11.2 refines the idea for multi-class settings
- Signaling weak class separation that a new label can sharpen
- This differs from absolute least-confidence alone by focusing on the competing pair of
- View files: ``modules/chapter11/example2/``

Example 11.2 — listing

```
"""Example 11.2 – margin sampling prioritizes the smallest top-two
gap."""

from __future__ import annotations

def margin(probs: dict[str, float]) -> float:
    """Return the gap between the top two class probabilities."""
    ordered = sorted(probs.values(), reverse=True)
    return ordered[0] - ordered[1]

def main() -> None:
    """Rank multi-class predictions by margin (smaller margin => query
    first)."""
    pool: list[tuple[str, dict[str, float]]] = [
        ("doc_clear", {"sports": 0.80, "politics": 0.12, "biz": 0.08}),
        ("doc_clear", {"sports": 0.70, "politics": 0.20, "biz": 0.10}),
        ("doc_clear", {"sports": 0.60, "politics": 0.30, "biz": 0.10}),
        ("doc_clear", {"sports": 0.50, "politics": 0.40, "biz": 0.10}),
        ("doc_clear", {"sports": 0.40, "politics": 0.50, "biz": 0.10}),
        ("doc_clear", {"sports": 0.30, "politics": 0.60, "biz": 0.10}),
        ("doc_clear", {"sports": 0.20, "politics": 0.70, "biz": 0.10}),
        ("doc_clear", {"sports": 0.10, "politics": 0.80, "biz": 0.10}),
        ("doc_clear", {"sports": 0.00, "politics": 0.90, "biz": 0.10}),
        ("doc_clear", {"sports": 0.00, "politics": 0.00, "biz": 0.00}),
    ]
    pool.sort(key=lambda x: margin(x[1]))
    for doc, probs in pool:
        print(doc, probs)
```


Example 11.3 — High-Entropy Multi-Class Prediction

- Example 11.3 — hands-on module
- Example 11.3 illustrates entropy sampling
- If the model predicts class A at 0.4 and classes B and C at 0.3 each
- Flat distributions across many classes are informative even when no single probability is
- View files: ``modules/chapter11/example3/``

Example 11.3 — listing

```
"""Example 11.3 – entropy sampling selects the flattest predictive
distribution."""

from __future__ import annotations

import math

def entropy(probs: list[float]) -> float:
    """Return Shannon entropy of a discrete probability
    distribution."""
    return -sum(p * math.log2(p) for p in probs if p > 0)

def main() -> None:
    """Compare entropy for peaked vs flat multi-class predictions."""
    cases: list[tuple[str, list[float]]] = [
        ("peaked (A 0.99)", [0.99, 0.05, 0.05]),
```

Active-learning pipeline steps

- The pipeline is incremental
- The loop stops when the model meets a performance threshold or the labeling budget is
- Effort concentrates on informative cases rather than easy majority examples

Example 11.4 — Active Learning for MRI Tumor Detection

- Example 11.4 — hands-on module
- Example 11.4 applies the full loop to expert MRI labeling
- A tumor detector starts from a small labeled MRI seed set
- Explore the chapter example module
- View files: ``modules/chapter11/example4/``

Takeaways

- Active learning is an iterative loop that queries humans where the model is least sure
- Uncertainty, margin, and entropy sampling are complementary ways to rank informativeness
- Medical and vision examples show how seed training plus selective expert labels can reduce

Next

- Complete the quiz for this part
- The next part covers complementary techniques beyond basic uncertainty

Learning objectives

- By the end of this part

Least confidence and related uncertainty measures

- Uncertainty sampling queries samples where the model is least predictable
- Least confidence focuses on near-ties such as 0.55 versus 0.45 in binary classification
- Margin sampling uses the top-two probability gap
- Entropy sampling uses the full output distribution
- These measures work well on complex boundaries but can skew the labeled set toward hard

Example 11.5 — Uncertain Chest X-ray Near Decision Boundary

- Example 11.5 — hands-on module
- Example 11.5 shows least-confidence selection on a near-tie X-ray prediction
- An X-ray scored 0.52 abnormal and 0.48 normal is queued for labeling because the near-tie
- Explore the chapter example module
- View files: ``modules/chapter11/example5/``

Diversity-based sampling

- Diversity-based sampling ensures selected samples represent the data distribution
- A common approach clusters unlabeled data and draws from underrepresented or highly
- The goal is coverage across scenarios so the model does not overfit a narrow subset of

Example 11.6 — Diversity Sampling Across Review Clusters

- Example 11.6 — hands-on module
- Example 11.6 uses clustering so sentiment labels cover mixed and neutral reviews
- Diversity-based sampling draws reviews from multiple clusters
- Explore the chapter example module
- View files: ``modules/chapter11/example6/``

Cost-sensitive active learning

- Cost-sensitive active learning incorporates labeling cost into selection
- A cost-impact model ranks samples by expected performance gain relative to annotation
- Under a fixed budget, the system selects the most impactful points within that budget
- This matters when some items need expensive experts and others can use cheaper annotators

Example 11.7 — Cost-Sensitive Routing in Legal Labeling

- Example 11.7 — hands-on module
- Example 11.7 allocates expensive expert labels only where they change the model most
- Maximizing accuracy gain per labeling dollar
- Explore the chapter example module
- View files: ``modules/chapter11/example7/``

Example 11.8 — Model-in-the-Loop Tumor Review

- Example 11.8 — hands-on module
- Model-in-the-loop annotation emphasizes human-machine collaboration
- The model proposes preliminary labels or highlights uncertain regions
- So later rounds need fewer human edits
- Explore the chapter example module
- View files: ``modules/chapter11/example8/``

Takeaways

- Least-confidence, margin
- Cost-sensitive routing protects scarce expert budgets
- Model-in-the-loop workflows let humans focus on corrections rather than labeling every

Next

- Complete the quiz for this part
- Dermoscopic lesion review, showing how uncertain items reach human experts

Learning objectives

- By the end of this part

Active learning in NLP

- Related tasks
- Annotating large corpora is expensive, especially for legal or medical text
- The model focuses labeling on informative or uncertain samples such as ironic

Example 11.9 — Active Learning for Review Sentiment

- Example 11.9 — hands-on module
- Example 11.9 queues uncertain reviews for human sentiment labels
- Improving nuance with fewer total annotations
- Explore the chapter example module
- View files: ``modules/chapter11/example9/``

Active learning in vision

- In computer vision
- Active learning focuses annotators on frames that most improve performance

Example 11.10 — Active Learning for Driving Perception

- Example 11.10 — hands-on module
- Example 11.10 focuses annotators on occluded or foggy driving scenes
- So perception models learn rare hard cases without labeling every fleet frame
- Explore the chapter example module
- View files: ``modules/chapter11/example10/``

Healthcare case study

- Medical image labeling requires domain expertise and is costly
- Active learning reduces the number of expert annotations while routing the most
- Experts spend limited time on atypical or overlapping pathologies rather than routine easy

Example 11.12 — Pneumonia Detection with Uncertain X-rays

- Example 11.12 — hands-on module
- Example 11.12 sends atypical chest X-rays to radiologists via active learning
- Improving edge-case detection with fewer labeled studies
- Explore the chapter example module
- View files: ``modules/chapter11/example12/``

Example 11.13 — Dermoscopic Lesion Uncertainty Queue

- Example 11.13 — hands-on module
- Example 11.13 prioritizes ambiguous skin lesions for dermatologist labels
- Active learning queues atypical or rare dermoscopic images for expert labeling so the
- Explore the chapter example module
- View files: ``modules/chapter11/example13/``

Takeaways

- Across NLP, driving perception
- Seed models plus selective expert review cut annotation volume while improving edge-case
- Routine easy items need not consume expert hours

Next

- Complete the quiz for this part
- The next part introduces weak supervision

Learning objectives

- By the end of this part

Definition of weak supervision

- Weak supervision trains models using data that is partially labeled
- Then managing the noise they introduce

Noisy, incomplete, and heuristic labels

- Noisy labels are incorrect or imprecise
- Incomplete labeling leaves much of the pool unlabeled and requires extrapolation from
- Heuristic or imprecise labels come from rules

Programmatic and data-programming ideas

- Weakly supervised pipelines often use labeling functions
- The key challenge is managing noise and conflict among sources while still extracting
- Later sections detail Snorkel-style aggregation

Advantages of weak supervision

- Weak supervision is cost-effective because it avoids labeling every item by hand
- It scales volume quickly when rules or distant sources can run over large corpora
- It also enables coverage in domains where clean labels are scarce

Sources of weak supervision

- Transfer from pretrained models when domain labels are limited
- Each source is imperfect

Distant supervision and related signals

- Distant supervision might infer company–investor relations from financial databases even
- Transfer learning can act as a weak signal when a general pretrained model is fine-tuned
- These patterns expand label volume but still require quality checks before production

Takeaways

- Weak supervision replaces full hand labeling with noisy, incomplete, or heuristic signals
- Advantages include cost, scale, and coverage when gold labels are scarce
- Sources range from rules and distant databases to pretrained models

Next

- Complete the quiz for this part
- Data programming that outputs probabilistic labels for training

Learning objectives

- By the end of this part

Programmatic labeling with Snorkel

- Snorkel is a framework for generating large-scale labeled datasets by combining weak
- Users write labeling functions
- A generative model estimates source accuracies and reconciles conflicts without requiring

Example 11.14 — Labeling Functions for Spam Detection

- Example 11.14 — hands-on module
- Example 11.14 sketches a Snorkel-style labeling-function workflow
- Snorkel estimates source accuracies and outputs probabilistic training labels
- Explore the chapter example module
- View files: ``modules/chapter11/example14/``

Example 11.14 — listing

```
"""Example 11.14 – heuristic labeling functions for weak spam
supervision."""

from __future__ import annotations

from collections import Counter

def lf_keywords(text: str) -> int:
    """Label +1 spam if promotional keywords appear, else abstain
    (0)."""
    keywords = ("free", "win", "prize", "click now")
    lowered = text.lower()
    return 1 if any(word in lowered for word in keywords) else 0
```

```
def lf_regex_money(text: str) -> int:
```

How conflicting labels are combined

- Each labeling function may cover only part of the data and may conflict with others
- Snorkel uses a probabilistic graphical model and expectation-maximization to refine
- The resulting probabilistic labels then train a discriminative model

Heuristic-based labeling

- Heuristic-based labeling writes if-then rules that automatically assign labels when
- Rules are often domain-specific and sparse: they cover some items well and miss edge cases
- Experts can validate a small subset and refine rules over time

Benefits and limits of heuristics

- Heuristics inject domain knowledge quickly and run at scale
- Limits include noise, sparse coverage, and brittleness on edge cases
- Heuristics therefore pair best with aggregation methods and small gold audits rather than

Data programming and probabilistic labels

- Data programming combines multiple weak sources into probabilistic labels rather than hard
- The method learns each labeling function's reliability
- Downstream models train on soft labels that reflect uncertainty instead of pretending

Takeaways

- Snorkel-style labeling functions turn heuristics and weak classifiers into aggregated
- Heuristic rules scale domain knowledge but need refinement and audits
- Data programming learns source quality and emits probabilistic labels that handle conflict

Next

- Complete the quiz for this part
- The next part introduces self-supervised learning

Learning objectives

- By the end of this part

Definition versus supervised learning

- Supervised learning trains on input–label pairs, such as images tagged cat or dog
- Self-supervised learning does not require manual task labels during pretraining
- Instead, the model creates its own supervisory signal from the data by solving a pretext

Why pretext tasks matter

- Pretext tasks force the model to learn useful features
- After pretraining
- This reduces labeling cost and can improve generalization when unlabeled corpora are

Vision pretext examples

- Rotation prediction of randomly rotated images
- Solving these proxies teaches spatial and semantic structure that later supports detection

Language pretext examples

- In natural language processing
- Next-sentence prediction and text infilling teach discourse and local coherence
- These tasks turn large unlabeled text corpora into representation learners before any

Contrastive learning idea

- Contrastive methods learn representations by pulling similar views of the same instance
- In vision, two augmented crops of one image form a positive pair
- The resulting encoder captures discriminative features that transfer well when a linear

Example 11.15 — Contrastive Pretraining Then Fine-Tune

- Example 11.15 — hands-on module
- Example 11.15 illustrates contrastive pretraining before fine-tuning
- A vision encoder is pretrained with SimCLR-style contrastive loss on unlabeled product
- Explore the chapter example module
- View files: ``modules/chapter11/example15/``

Takeaways

- Self-supervised learning creates supervision from unlabeled structure via pretext tasks
- It sits between supervised and unsupervised learning
- Contrastive pretraining plus light fine-tuning is a practical recipe when downstream

Next

- Complete the quiz for this part
- Time-series forecasting, emphasizing data efficiency when downstream labels remain scarce

Learning objectives

- By the end of this part

Computer vision applications

- Segmentation before scarce labels are applied
- Which matters in domains where bounding boxes or masks are expensive

Data efficiency in vision

- Once representations exist
- Teams still validate on held-out gold labels

Natural language processing applications

- Masked language modeling and related objectives pretrain encoders on unlabeled text
- Downstream tasks such as classification, named entity recognition
- Large web-scale corpora supply the unlabeled fuel for that pretraining stage

Transfer to specialized text

- Domain teams often start from a general pretrained language model and fine-tune on a
- The pretext-learned context and syntax transfer
- This pattern is common when expert text labels are scarce

Time-series forecasting and imputation

- For time series
- Models learn temporal structure from unlabeled sensor or transaction streams
- Gap filling also prepares incomplete series for downstream monitoring tasks

Anomaly detection and contrastive sequences

- Useful for fraud or equipment failure
- Clustering without labeling every window

Takeaways

- Self-supervised applications span vision, language, and time series with a shared theme
- Benefits include data efficiency, real-time monitoring potential
- Quality still depends on validating the downstream labeled evaluation set

Next

- Complete the quiz for this part
- The next part compares crowdsourcing and expert annotation on scale, cost, turnaround

Learning objectives

- By the end of this part

Crowdsourcing advantages

- For annotation, crowdsourcing offers scale, lower per-item cost, fast turnaround
- Those advantages matter most when the schema is simple, guidelines are clear
- This section emphasizes when experts must own labels rather than restating platform

Example 11.16 — Crowd Scale for Attribute Labels

- Example 11.16 — hands-on module
- Example 11.16 shows crowd scale for high-volume attribute labeling
- A vision team needs millions of face images tagged with age band, expression
- Crowdsourcing platforms distribute the schema across many workers so labeling finishes in
- Explore the chapter example module
- View files: ``modules/chapter11/example16/``

Example 11.17 — Crowd Cost for Product Images

- Example 11.17 — hands-on module
- Example 11.17 contrasts crowd cost with expert-only product labeling
- A retailer needs thousands of product photos labeled for category and attributes
- Paying only domain experts would blow the budget
- Explore the chapter example module
- View files: ``modules/chapter11/example17/``

Crowdsourcing challenges

- Crowd workers may be inconsistently trained or motivated, producing noisy annotations
- Mitigation requires ground-truth checks, gold questions, and redundancy
- Platform ethics and inter-annotator agreement controls from earlier chapters should be in

Example 11.20 — Expert Quality for Radiology Labels

- Example 11.20 — hands-on module
- Expert annotation uses specialists for tasks that need deep domain knowledge
- Example 11.20 shows why expert quality matters for clinical images
- Explore the chapter example module
- View files: ``modules/chapter11/example20/``

Example 11.22 — Legal Experts for Contract Clauses

- Example 11.22 — hands-on module
- Experts also improve consistency on complex legal schemas
- Example 11.22 keeps legal clause labeling with domain experts
- Explore the chapter example module
- View files: ``modules/chapter11/example22/``

Expert challenges

- Expert annotation is expensive and hard to scale because skilled professionals are scarce
- Careful analysis also slows throughput
- Which motivates hybrid designs in the next part

Takeaways

- Crowds excel at scale, cost, and speed on clear schemas with gold checks
- Experts excel at precision and consistency on radiology, legal
- Expert-only staffing hits cost and capacity walls

Next

- Complete the quiz for this part
- The next part covers hybrid annotation pipelines

Learning objectives

- Speed benefits of hybrid staffing

Why hybrid approaches

- Hybrid approaches balance cost, speed
- A useful pattern invests scarce experts where schema and gold labels are defined

Example 11.25 — Expert Gold Then Crowd Scale-Up

- Example 11.25 — hands-on module
- Example 11.25 seeds crowd work with an expert gold set
- Experts first label a small medical-image gold set
- Crowds then label the remainder
- Explore the chapter example module
- View files: ``modules/chapter11/example25/``

Example 11.26 — Crowd Objects Expert Pathology

- Example 11.26 — hands-on module
- Example 11.26 splits simple objects to the crowd and pathology to experts
- Crowd workers tag cars and trees in street scenes
- Difficulty, not platform preference alone, drives who sees which item
- Explore the chapter example module
- View files: ``modules/chapter11/example26/``

Active learning in the hybrid queue

- Active learning can sit inside the hybrid workforce
- A model trained on a small expert seed set queries the most uncertain items for experts
- That routing shrinks expert hours while protecting hard cases that would otherwise pollute

Example 11.28 — Active Learning Routes Hard Cases

- Example 11.28 — hands-on module
- Example 11.28 routes uncertain NLP items to experts via active learning
- Shrinking expert hours while protecting hard cases
- Explore the chapter example module
- View files: ``modules/chapter11/example28/``

Example 11.29 — Hybrid Cost Savings Pattern

- Example 11.29 — hands-on module
- Hybrid benefits include cost efficiency
- Example 11.29 summarizes the hybrid cost pattern
- Explore the chapter example module
- View files: ``modules/chapter11/example29/``

Quality, speed, and looking ahead

- Hybrid quality control uses expert oversight of crowd batches and guideline updates when
- Crowds deliver overnight throughput on easy items while a smaller expert pool clears the
- Move next to scalable storage and lineage for large labeled corpora

Takeaways

- Hybrid pipelines assign routine volume to crowds and reserve experts for gold
- Active learning can route uncertainty to the right workforce tier
- Cost, quality

Next

- Complete the quiz for this part
- This chapter's clip sequence is complete